

- Chapitre 2 -

Quelques algorithmes de parcours d'un graphe

I. L'algorithme de parcours en largeur d'un graphe, ou BFS

Le parcours en largeur d'un graphe consiste à le parcourir « étage par étage » : partant d'un sommet « source », on découvre tous les voisins de ce sommet (on se trouve à 1 arête de distance du sommet source, soit à l'étage 1), puis on découvre tous les nouveaux voisins des sommets de l'étage 1 (ces sommets se trouvent à « l'étage 2 » : ils sont à une distance de 2 arêtes du sommet source), etc...

Ce parcours permet de déterminer tous les sommets accessibles depuis le sommet source. De plus, le parcours en largeur construit, pour tout sommet s accessible depuis le sommet source s_0 , un chemin de longueur minimale, c'est-à-dire admettant $d(s_0, s)$ arêtes.

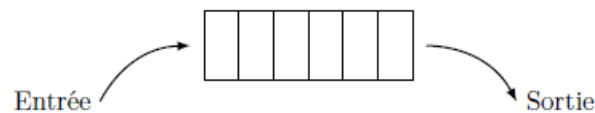
Les lettres BFS signifient Breadth First Search.

I.1. Présentation d'une file en informatique

Pour le parcours en largeur d'un graphe (orienté ou non), on utilise « une file ».

Une file est un objet informatique qui permet de stocker des éléments (de type quelconque), sous la forme d'une liste par exemple, en suivant le principe **FIFO** : **Fist in, First out**.

Représentation d'une file en informatique :



Le mot « file » est cohérent avec sa signification : penser à une file de voitures à un feu rouge, à la file d'attente à la caisse d'un magasin, etc

Dans les algorithmes que nous utiliserons, nous représenterons les files sous forme d'une variable Python de type `list`, et ainsi :

- pour ajouter un élément x à la file L , on utilisera la commande :

```
L.insert(0,x)
```

- pour enlever un élément à la file L , et affecter la valeur de cet élément à une variable u , on utilisera la commande :

```
u=L.pop()
```

Avec les deux commandes ci-dessus, les éléments d'une file sont bien ajoutés à gauche de la liste, et on les supprime par le côté droit de la liste.

I.2. Parcours d'un graphe en largeur

On cherche ici à donner un algorithme qui, à partir d'un graphe G (orienté ou non) et d'un sommet s de G , renvoie la liste des sommets de G reliés à s (par un chemin ou une chaîne), à l'aide d'un parcours en largeur des sommets du graphe G , en partant du « sommet source » s .

Algorithme de parcours en largeur

Soit $G = (S, A)$ un graphe et $s \in S$.

On explique ici l'algorithme de parcours BFS des sommets de G , en partant du sommet source s , afin d'obtenir la liste des sommets de G accessibles par une chaîne ou un chemin ayant pour origine s .

- (1) On dispose de deux listes, initialement vides :
l'une, notée **Access**, donnant la liste des sommets accessibles dans G à partir du sommet s ,
l'autre, une file notée **Avisiter**, donnant la liste des sommets accessibles à partir de s et déjà découverts et dont il faut ensuite découvrir les voisins ou successeurs.
- (2) On colore en blanc tous les sommets de G : on signale que pour l'instant, aucun sommet n'a été visité.
- (3) On ajoute dans **Access** le sommet s .
On affecte au sommet s la couleur grise : le sommet s a été découvert, et on doit rechercher ses successeurs (ou ses voisins).
- (4) On recherche tous les successeurs (ou voisins) de s colorés en blanc :
→ ils sont ajoutés à la liste **Access**, puis ils sont colorés en gris,
→ ils sont ajoutés dans la file **Avisiter**
- (5) On colore en noir le sommet s : on signale que l'étude du sommet s est finie puisque tous ses successeurs (ou voisins) ont été découverts.
- (6) Si la liste **Avisiter** est vide, le parcours de G est fini.
Sinon, on enlève de la file **Avisiter** son élément de droite (le premier sommet découvert) qui devient notre nouveau sommet s à étudier : on reprend l'algorithme au point 4.

Pour colorer les sommets, on pourra utiliser une troisième liste **couleur** qui évoluera au fur et à mesure de l'algorithme. Cette liste **couleur** peut aussi être omise pour l'écriture de l'algorithme.

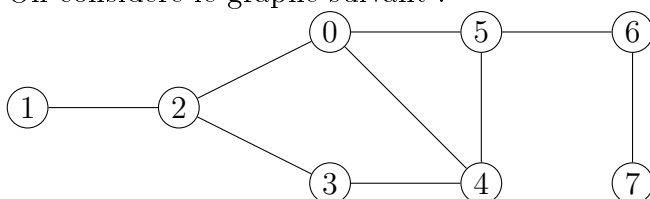
Remarque :

Supposons que G est un graphe d'ordre n , dont les sommets sont numérotés par des entiers consécutifs.

Par convention, à chaque étape de l'algorithme, les sommets blancs voisins du sommet en cours d'étude entreront dans la file **Avisiter** dans l'ordre croissant de leur numéro.

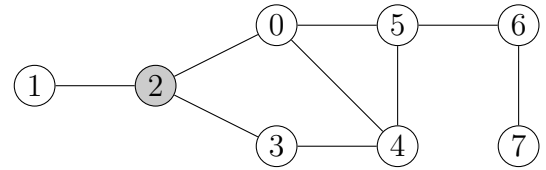
Exemple :

On considère le graphe suivant :

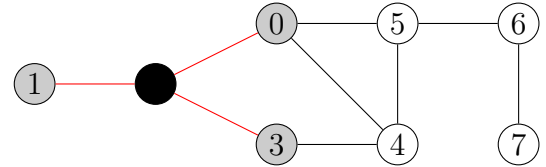


A partir du sommet source 2, l'algorithme BFS parcourt le graphe dans l'ordre suivant :

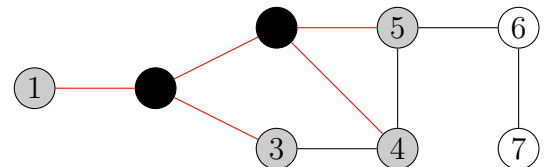
Début de l'algorithme :
on va rechercher tous les voisins de 2.
On initialise **Access** à **Access=[2]**.



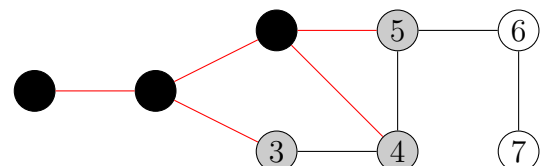
→ On détermine tous les voisins de 2 :
Access devient [3,1,0,2],
Avisiter devient [3,1,0].
→ L'étude du sommet 2 est finie : on colorie le sommet 2 en noir.



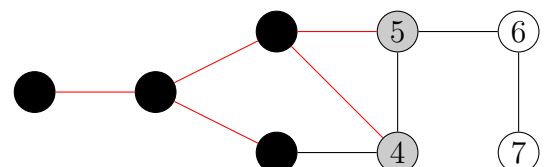
On étudie maintenant le sommet 0 :
→ on enlève le sommet 0 de la file
Avisiter :
Avisiter devient [3,1],
→ on détermine les voisins colorés en blanc de 0 :
Access devient [5,4,3,1,0,2],
Avisiter devient [5,4,3,1].
→ L'étude du sommet 0 est finie : on colorie le sommet 0 en noir.



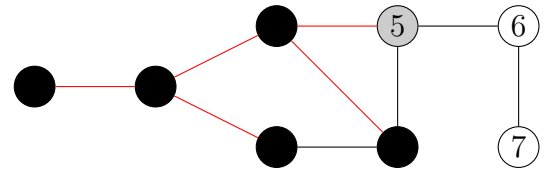
On étudie maintenant le sommet 1 :
→ on enlève le sommet 1 de la file
Avisiter :
Avisiter devient [5,4,3],
→ on détermine les voisins colorés en blanc de 1 :
Access et **Avisiter** sont inchangées.
→ L'étude du sommet 1 est finie : on colorie le sommet 1 en noir.



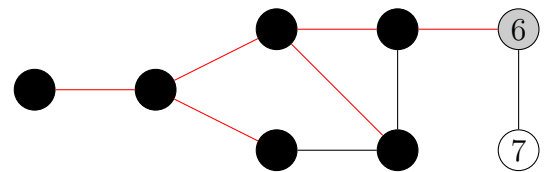
On étudie maintenant le sommet 3 :
→ on enlève le sommet 3 de la file
Avisiter :
Avisiter devient [5,4],
→ on détermine les voisins colorés en blanc de 3 :
Access et **Avisiter** sont inchangées.
→ L'étude du sommet 3 est finie : on colorie le sommet 3 en noir.



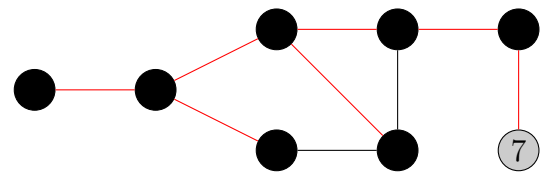
On étudie maintenant le sommet 4 :
 → on enlève le sommet 4 de la file
Avisiter :
Avisiter devient [5],
 → on détermine les voisins colorés en blanc de 4 :
Access et **Avisiter** sont inchangées.
 → L'étude du sommet 4 est finie : on colorie le sommet 4 en noir.



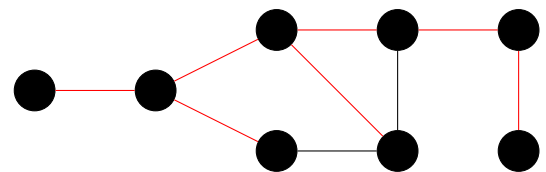
On étudie maintenant le sommet 5 :
 → on enlève le sommet 5 de la file
Avisiter :
Avisiter devient [],
 → on détermine les voisins colorés en blanc de 5 :
Access devient [6,5,4,3,1,0,2],
Avisiter devient [6].
 → L'étude du sommet 5 est finie : on colorie le sommet 5 en noir.



On étudie maintenant le sommet 6 :
 → on enlève le sommet 6 de la file
Avisiter :
Avisiter devient [],
 → on détermine les voisins colorés en blanc de 6 :
Access devient [7,6,5,4,3,1,0,2],
Avisiter devient [7].



On étudie maintenant le sommet 7 :
 → on enlève le sommet 7 de la file
Avisiter :
Avisiter devient [],
 → on détermine les voisins colorés en blanc de 7 :
Access et **Avisiter** restent inchangées.
 → L'étude du sommet 7 est finie : on colorie le sommet 7 en noir.



L'algorithme de parcours en largeur est alors terminé : la liste **Avisiter** est vide.
 Notons qu'à l'issue de l'algorithme tous les sommets du graphe sont colorés en noir. Cela signifie que tout sommet du graphe G peut être relié par une chaîne au sommet source 2.
 Le parcours rouge effectué par l'algorithme BFS est un arbre (graphe connexe, sans cycle).
 On dit qu'il s'agit d'un arbre « couvrant » car cet arbre contient tous les sommets de G .

Parcours en largeur, avec la commande Python `nx.bfs_tree`

On suppose le module Python `networkx` importé sous la forme : `import networkx as nx`. On suppose que `G` est une variable Python représentant un graphe et définie par la fonction Python `nx.Graph` ou `nx.DiGraph`, et que `s` est l'un des sommets du graphe `G`. La commande Python

```
parcours_larg = nx.bfs_tree(G, source=s)
```

renvoie le graphe orienté constitué du sommet `s` et des sommets accessibles depuis le sommet source `s`, l'orientation découlant de l'ordre de leur découverte, obtenu par l'algorithme de parcours en largeur, autrement dit par l'algorithme BFS.

Remarque :

Notons que la commande Python `nx.bfs_tree(G, source=s)` inclut le sommet source `s` dans la liste des sommets accessibles à partir du sommet `s`. Dans le cas d'un graphe orienté, cette commande ne permet pas de décider si il existe ou non une boucle (s, s) dans le graphe `G`.

Exemple :

On reprend l'exemple précédent.

On considère le code Python ci-dessous, définissant le graphe `G` étudié.

```
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt

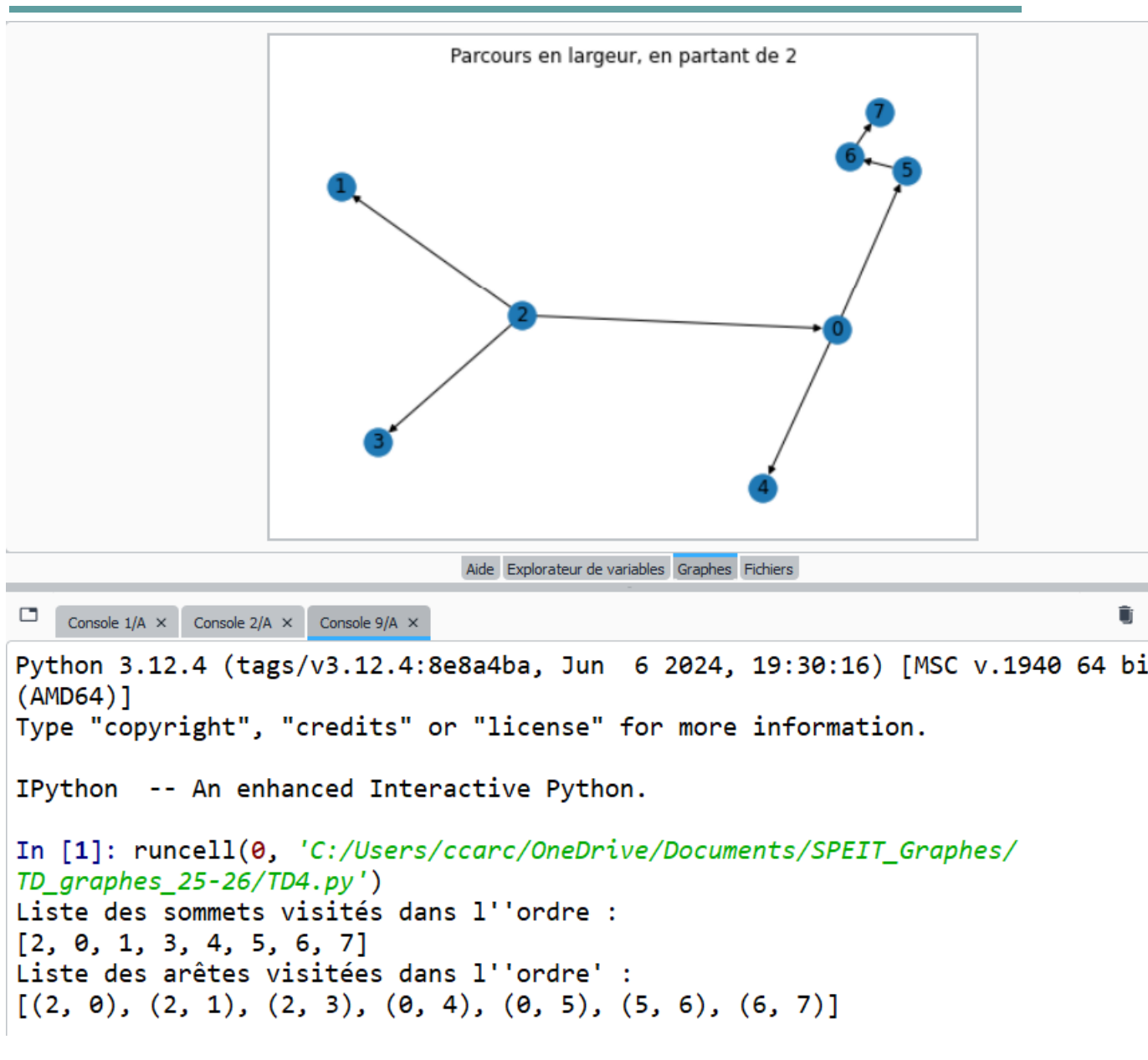
D={0:[2,4,5],1:[2],2:[0,1,3],\
   3:[2,4],4:[0,3,5],5:[0,4,6],6:[5,7],7:[6]}

G=nx.Graph(D)
s=2
parcours_larg = nx.bfs_tree(G, source=s)

print("Liste des sommets visités dans l'ordre : ")
print(list(parcours_larg.nodes()))
print("Liste des arêtes visitées : ")
print(list(parcours_larg.edges()))

nx.draw(parcours_larg, with_labels=True)
plt.title('Parcours en largeur, en partant de '+repr(s))
plt.show()
```

Après exécution des commandes, on obtient l'écran de sortie ci-après :



Exercice 1 :

Soit $G = (S, A)$ un graphe non orienté.

Soit $s \in S$.

1. On rappelle que la relation « être connectés », notée $\sim$, définit une relation d'équivalence sur S .

Montrer que l'ensemble des sommets constituant la liste des sommets accessibles depuis s obtenue par l'algorithme BFS appliqué au graphe G et au sommet source s , est la classe d'équivalence de s .

2. On suppose dans cette questions que G est connexe.

- (a) Soit t un sommet de G distinct de s .

Expliquer comment obtenir la distance $d(s, t)$ à l'aide de l'algorithme BFS de parcours de G avec pour sommet source s .

- (b) On considère le parcours en largeur du graphe G , de sommet source s .

A chaque étape de l'algorithme, on recherche les voisins colorés en blanc d'un

sommet particulier, noté u , et lorsqu'un nouveau sommet v est découvert, on colore en rouge l'arête $\{u, v\}$.

A l'issue de l'algorithme, tous les sommets de G ont été découverts, car G est connexe, certaines des arêtes de G sont colorées en rouge et d'autres non.

On note R l'ensemble des arêtes de G colorées en rouge.

Montrer que le graphe $H = (S, R)$ est un arbre.

II. L'algorithme de parcours en profondeur, ou DFS

Comme pour l'algorithme BFS, l'algorithme DFS parcourt un graphe donné à partir d'un sommet source s_0 , et détermine tous les sommets accessibles à partir du sommet source s_0 .

Le parcours en profondeur d'un graphe construit, en partant d'un sommet source s_0 , la plus longue chaîne (ou chemin) élémentaire possible. Puis, lorsque cette chaîne est construite, on remonte la chaîne et on s'arrête au premier sommet qui admet des voisins non encore découverts, et ce sommet devient le nouveau sommet source...

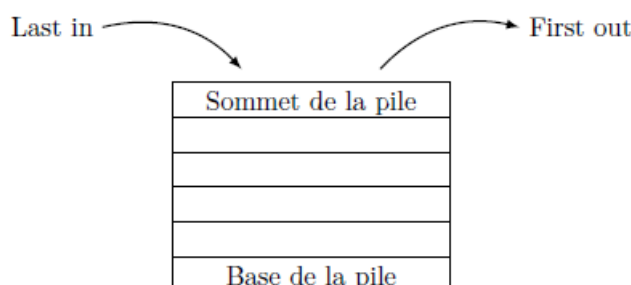
Les lettres DFS signifient Depth First Search.

II.1. Présentation d'une pile en informatique

Pour le parcours en profondeur d'un graphe (orienté ou non), on utilise « une pile ».

Une pile est un objet informatique qui permet de stocker des éléments (de type quelconque), sous la forme d'une liste par exemple, en suivant le principe **LIFO** : **Last in, First out**.

Représentation d'une pile en informatique :



Le mot « pile » est cohérent avec sa signification : penser à une pile de livres, une pile d'assiettes, etc

Dans les algorithmes que nous utiliserons, nous représenterons les piles sous forme d'une variable Python de type `list`, et ainsi :

- pour ajouter un élément x à la pile L , on utilise la commande :

```
L=L . append ( x )
```

- pour enlever un élément à la pile L , on utilise la commande :

```
u=L . pop ( )
```

Avec les deux commandes ci-dessus, les éléments d'une pile sont ajoutés à droite de la liste, et on les supprime par le côté droit de liste.

II.2. Parcours d'un graphe en profondeur

On cherche ici à donner un algorithme qui, à partir d'un graphe G (orienté ou non) et d'un sommet s_0 de G renvoie la liste des sommets de G reliés à s_0 (par une chaîne ou un chemin), à l'aide d'un parcours en profondeur des sommets du graphe G , en partant du « sommet source » s_0 .

Le parcours en profondeur du graphe G , en partant du sommet source s_0 , consiste à chercher un voisin (ou successeur) de s_0 , puis à continuer la recherche en profondeur, en cherchant un voisin de ce voisin, etc, jusqu'à ne plus trouver de voisin (ou successeur). On remonte alors la liste des sommets découverts (qui constitue un arbre) et dès qu'un sommet a un voisin (ou successeur) non

encore découvert, on recommence : on donne un successeur de ce sommet, puis on cherche un successeur de ce successeur, etc

On construit ainsi un chemin le plus long possible à partir du sommet source s_0 , une fois le chemin terminé, on remonte le long de ce chemin jusqu'à trouver un sommet admettant un voisin non découvert, et on construit à partir de ce sommet un nouveau chemin dans G le plus long possible parmi les sommets non encore explorés, etc

Algorithme de parcours en profondeur

Soit $G = (S, A)$ un graphe et $s_0 \in S$.

On explique ici l'algorithme de parcours en profondeur des sommets de G , en partant du sommet source s_0 , afin d'obtenir la liste des sommets de G accessibles par une chaîne ou un chemin en partant de s_0 .

- (1) On dispose de deux listes, initialement vides :
l'une, notée **Access**, donnant la liste des sommets accessibles dans G à partir du sommet s_0 ,
l'autre, une pile notée **Avisiter**, donnant la liste des sommets accessibles à partir de s_0 et déjà découverts et dont il faut ensuite découvrir les voisins ou successeurs.
- (2) On colore en blanc tous les sommets de G : on signale que pour l'instant, aucun sommet n'a été visité.
- (3) On ajoute dans **Access** et dans **Avisiter** le sommet s_0 .
On affecte au sommet s_0 la couleur grise : le sommet s_0 a été découvert, et on doit rechercher ses successeurs (ou ses voisins).
- (4) On commence l'algorithme de parcours en profondeur :
 - on dépile l'élément du haut de la pile **Avisiter**, noté s (initialement, il s'agit du sommet $s = s_0$) : on va chercher un voisin de s ,
 - on rempile l'élément s dans la pile **Avisiter** (le sommet s peut avoir encore d'autres voisins à découvrir),
 - si s a un voisin (ou successeur) coloré en blanc, noté t :
 - ce sommet voisin t est ajouté à la liste **Access**, et on colore ce sommet t en gris,
 - ce sommet t est ajouté dans la pile **Avisiter**.
 - si s n'a pas de voisin (ou successeur) coloré en blanc, l'étude du sommet s est finie :
 - on dépile l'élément s de la pile **Avisiter**, et on colore ce sommet s en noir.
- (5) On continue le parcours en profondeur :
on reprend l'algorithme de parcours au point précédent (4).
- (6) L'algorithme s'arrête lorsque la liste **Avisiter** est vide.

Pour colorer les sommets, on pourra utiliser une troisième liste **couleur** qui évoluera au fur et à mesure de l'algorithme. Cette liste **couleur** peut aussi être omise pour l'écriture de l'algorithme.

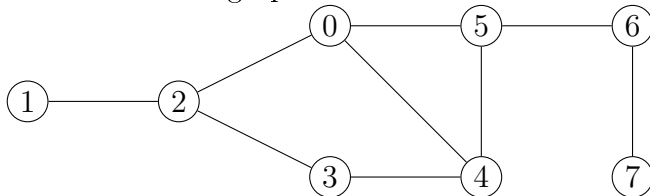
Remarque :

- À chaque étape de l'algorithme, on cherche un sommet de couleur blanche, voisin du sommet s (le sommet du haut de la pile **Avisiter**). Parmi tous les voisins blancs de s , on choisit celui de valeur minimale, lorsque les sommets de G sont représentés par des entiers.
- Comme pour l'algorithme de parcours BFS, on cherche tous les sommets accessibles depuis le sommet source s_0 de G , et on colore au fur et à mesure les sommets visités au cours de

l'algorithme afin de ne pas « repasser deux fois au même endroit ».

Exemple :

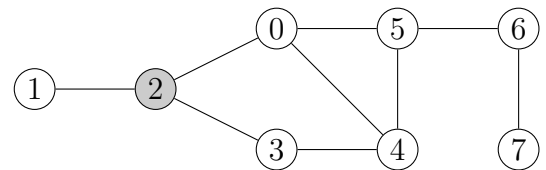
On considère le graphe suivant :



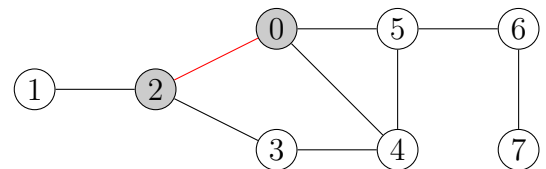
A partir du sommet source 2, l'algorithme DFS parcourt le graphe dans l'ordre suivant :

Début de l'algorithme :

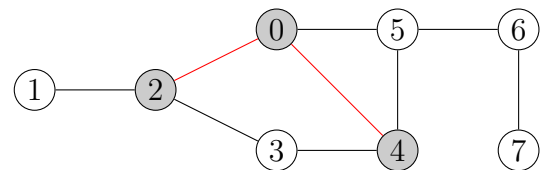
Access devient [2],
on colorie 2 en gris, et **Avisiter** devient [2].



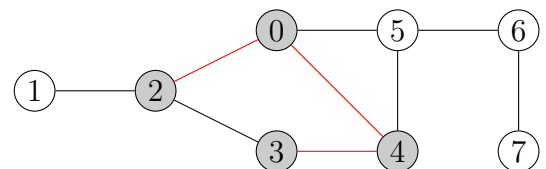
On détermine un voisin blanc de 2 :
on trouve 0, que l'on colorie en gris.
Access devient [2,0],
Avisiter devient [2,0],



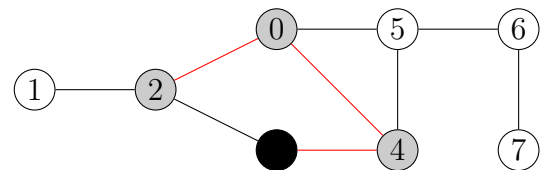
On cherche maintenant un voisin blanc du
sommet 0 :
on trouve 4, que l'on colorie en gris.
Access devient [2,0,4].
Avisiter devient [2,0,4].



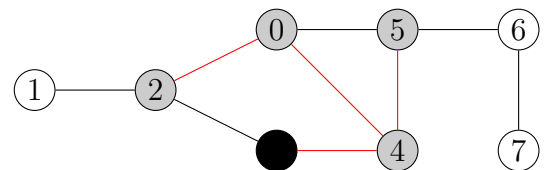
On cherche maintenant un voisin blanc du
sommet 4 :
on trouve 3, que l'on colorie en gris.
Access devient [2,0,4,3].
Avisiter devient [2,0,4,3].



On cherche maintenant un voisin blanc du
sommet 3 : il n'y en a pas.
L'étude du sommet 3 est finie :
→ on le colorie en noir,
→ on le dépile de la pile **Avisiter** qui
devient [2,0,4]
→ on s'intéresse maintenant au nouveau
sommet en haut de la pile : 4.



On cherche maintenant un voisin blanc du
sommet 4 : on trouve 5, que l'on colorie en
gris.
Access devient [2,0,4,3,5].
Avisiter devient [2,0,4,5].

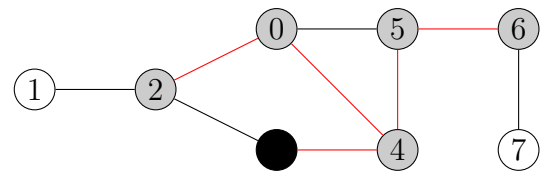


On cherche maintenant un voisin blanc du sommet 5 :

on trouve 6, que l'on colorie en gris.

Access devient $[2,0,4,3,5,6]$.

Avisiter devient $[2,0,4,5,6]$.

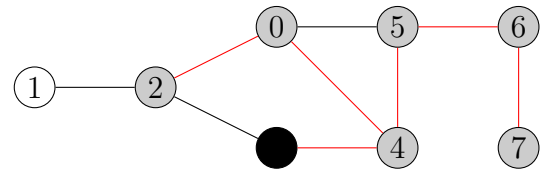


On cherche maintenant un voisin blanc du sommet 6 :

on trouve 7, que l'on colorie en gris.

Access devient $[2,0,4,3,5,6,7]$.

Avisiter devient $[2,0,4,5,6,7]$.



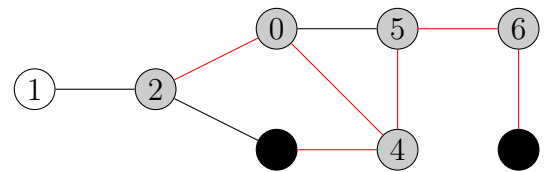
On cherche maintenant un voisin blanc du sommet 7 : il n'y en a pas.

L'étude du sommet 7 est finie :

→ on le colorie en noir,

→ on le dépile de la pile Avisiter qui devient $[2,0,4,5,6]$

→ on s'intéresse maintenant au nouveau sommet en haut de la pile : 6.



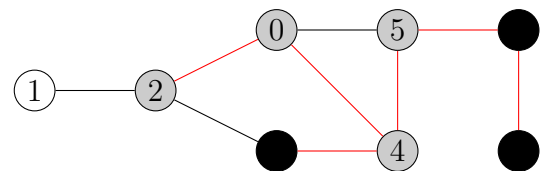
On cherche maintenant un voisin blanc du sommet 6 : il n'y en a pas.

L'étude du sommet 6 est finie :

→ on le colorie en noir,

→ on le dépile de la pile Avisiter qui devient $[2,0,4,5]$

→ on s'intéresse maintenant au nouveau sommet en haut de la pile : 5.



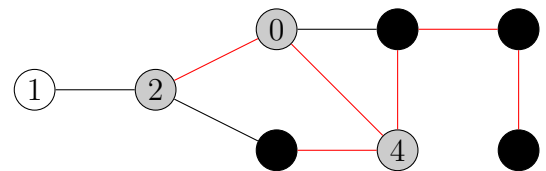
On cherche maintenant un voisin blanc du sommet 5 : il n'y en a pas.

L'étude du sommet 5 est finie :

→ on le colorie en noir,

→ on le dépile de la pile Avisiter qui devient $[2,0,4]$

→ on s'intéresse maintenant au nouveau sommet en haut de la pile : 4.



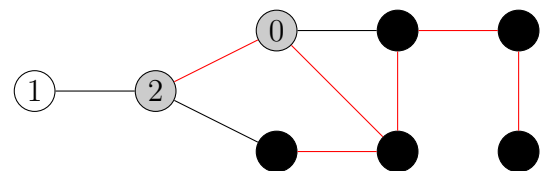
On cherche maintenant un voisin blanc du sommet 4 : il n'y en a pas.

L'étude du sommet 4 est finie :

→ on le colorie en noir,

→ on le dépile de la pile Avisiter qui devient $[2,0]$

→ on s'intéresse maintenant au nouveau sommet en haut de la pile : 0.



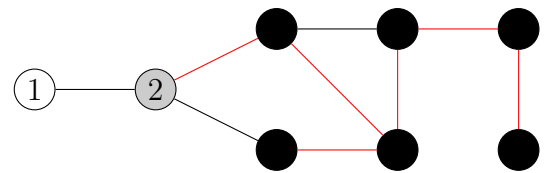
On cherche maintenant un voisin blanc du sommet 0 : il n'y en a pas.

L'étude du sommet 0 est finie :

→ on le colorie en noir,

→ on le dépile de la pile **Avisiter** qui devient [2]

→ on s'intéresse maintenant au nouveau sommet en haut de la pile : 2.

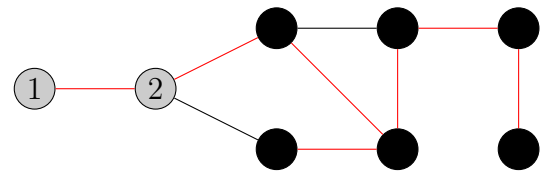


On cherche maintenant un voisin blanc du sommet 2 :

on trouve 1, que l'on colorie en gris.

Access devient [2,0,4,3,5,6,7,1].

Avisiter devient [2,1].



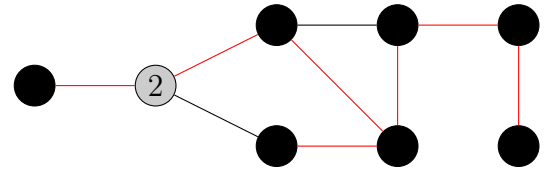
On cherche maintenant un voisin blanc du sommet 1 : il n'y en a pas.

L'étude du sommet 1 est finie :

→ on le colorie en noir,

→ on le dépile de la pile **Avisiter** qui devient [2]

→ on s'intéresse maintenant au nouveau sommet en haut de la pile : 2.



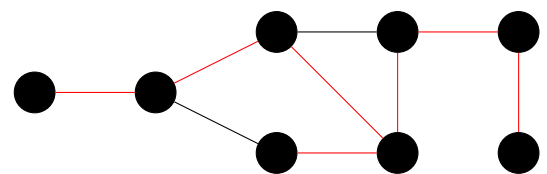
On cherche maintenant un voisin blanc du sommet 2 : il n'y en a pas.

L'étude du sommet 2 est finie :

→ on le colorie en noir,

→ on le dépile de la pile **Avisiter** qui devient []

La pile **Avisiter** étant vide, l'algorithme est fini et **Access** contient la liste de tous les sommets de G accessibles depuis le sommet source 2.



Notons qu'à l'issue de l'algorithme tous les sommets du graphe sont colorés en noir. Cela signifie que tout sommet du graphe G peut être relié par une chaîne au sommet source 2.

Le parcours rouge effectué par l'algorithme DFS est un arbre (graphe connexe, sans cycle).

On dit qu'il s'agit d'un arbre « couvrant » car cet arbre contient tous les sommets de G .

Parcours en profondeur, avec la commande Python `nx.dfs_tree`

La commande Python

```
parcours_prof = nx.dfs_tree(G, source=s)
```

renvoie le graphe orienté constitué du sommet s et des sommets accessibles depuis le sommet source s , l'orientation découlant de l'ordre de leur découverte, obtenu par l'algorithme de parcours en profondeur.

Exemple :

On reprend l'exemple précédent.

On considère le code Python ci-dessous, définissant le graphe G représenté.

```
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt

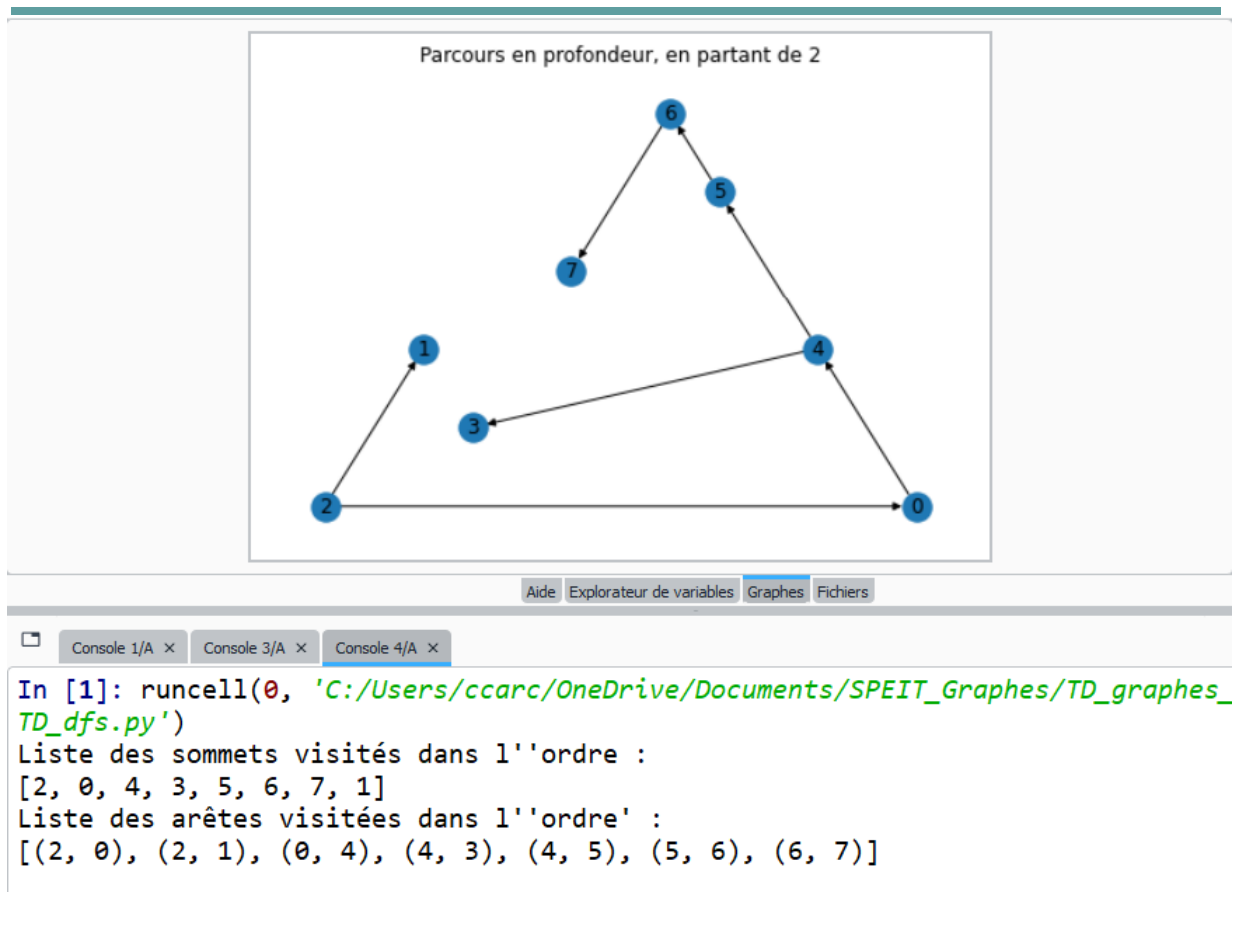
D={0:[2,4],1:[2],2:[0,1,3],\
   3:[2,4],4:[0,5],5:[0,4,6],6:[5,7],7:[6]}

G=nx.Graph(D)
s=2
parcours_prof = nx.dfs_tree(G, source=s)

print("Liste des sommets visités dans l'ordre : ")
print(list(parcours_prof.nodes()))
print("Liste des arêtes visitées : ")
print(list(parcours_prof.edges()))

plt.clf()
position=nx.planar_layout(parcours_prof)
nx.draw(parcours_prof, with_labels=True,pos=position)
plt.title('Parcours en largeur, en partant de '+repr(s))
plt.show()
```

Après exécution des commandes, on obtient l'écran de sortie ci-après :



Remarque :

Comme pour l'algorithme BFS, l'algorithme de parcours en profondeur DFS d'un graphe non orienté, partant d'un sommet source s_0 , construit un arbre dont les sommets sont les éléments de la composante connexe dans G de s .